

ATTENZIONE! QUESTO MATERIALE DIDATTICO È PER USO PERSONALE DELLO STUDENTE
ED È COPERTO DA COPYRIGHT. NE È VIETATA LA RIPRODUZIONE O IL RIUTILIZZO ANCHE PARZIALE,
AI SENSI E PER GLI EFFETTI DELLA LEGGE SUL DIRITTO D'AUTORE.

Verifica statica

Ingegneria del Software Avanzata

Università di Ferrara

CdLM in Ingegneria Informatica e dell'Automazione

A.A. 2019/2020

Marco Alberti (marco.alberti@unife.it)

Analisi (o verifica statica)

- Tecnica statica (non richiede esecuzione del programma)
- Vantaggio: il risultato vale non per una singola esecuzione (come il test), ma per una *classe* di esecuzioni.
- Svantaggi:
 - spesso soggetta a fallibilità umana
 - opera su un modello del sistema anziché sul sistema stesso
- Tecniche
 - informali (valutano qualità dell'artefatto)
 - formali (valutano proprietà matematiche dell'artefatto)

Analisi informale

Analisi informale

- Analisi di
 - specifiche dei requisiti
 - specifica di progetto
 - codice
- e, se applicabile, simulazione manuale del comportamento
- Non dovrebbe eseguirla (solo) l'autore dell'artefatto.
- Sul codice:
 - code walkthroughs (esecuzioni manuali)
 - code reviews (ispezioni alla ricerca di errori o regole di codifica non rispettate)
- Alcune di queste analisi sono automatizzate in IDE

Code reviews

Difetti comuni (non intercettati dal compilatore):

- variabili non inizializzate (analisi di flusso)
- assegnamenti incompatibili
- cicli che non terminano
- errori 0/1
- indici di array fuori campo
- errori su allocazione dinamica di memoria (es. uso dopo free)
- mancata corrispondenza fra parametri formali e attuali
- confronti di uguaglianza fra valori float

Code reviews

- Rispetto delle regole di codifica, come
 - Divieto di uso di determinati costrutti (es. cast implicito in C)
 - Leggibilità (es. convenzioni sui nomi delle variabili)
 - Testabilità (es. minimizzazione delle dipendenze implicite)
 - Conformità del codice al progetto (es. relazioni fra moduli)
 - Sicurezza (es. divieto di uso di scanf)
- La frequenza e le modalità delle sessioni dipendono dal prodotto e dal processo.
- Svolte (anche) da persone diverse dall'autore.
- Classe di programmi che eseguono (alcuni) controlli: linter

Prove formali di correttezza

Prove formali di correttezza

- Presuppongono specifica di un frammento di codice data tramite precondizioni e postcondizioni
- Dimostrazioni matematiche che la postcondizione discende dalla precondizione e dal programma

```
{true}
1 begin
2     read (a); read (b);
3     x := a + b;
4     write (x);
5 end
{output = input1 + input2}
```

- Postcondizione vera se, prima di 4,
- $x = \text{input1} + \text{input2}$, vero se, prima di 3,
- $a + b = \text{input1} + \text{input 2}$, vera per la semantica di read.

Backward substitution

- Procedimento che costruisce una catena di implicazioni inverse dalla postcondizione alla preconditione
- Data un'istruzione
 $a := \text{exp}$,
una sua postcondizione Post vale dopo l'esecuzione se, prima dell'esecuzione, valeva l'espressione ottenuta sostituendo, in Post , exp a ogni occorrenza di a .
- Esempio:
 $\{x = 5\}$
 $x := x + 1$
 $\{x = 6\}$
- La postcondizione vale dopo l'esecuzione se, prima dell'esecuzione, valeva $x+1=6$, cioè $x=5$ (precondizione)

Esempio

$$\{y < a + b - 50\}$$
$$z := a + b;$$
$$x := z - 43$$
$$\{x > y + 7\}$$

Con backward substitution

- $\{z - 43 > y + 7\}$ (postcond. di $z:=a+b$)
- $\{a + b - 43 > y + 7\}$ (equivalente alla preconditione)

Input-output

- input e output come file (sequenze di valori)
- `read(x)` trattata come assegnamento da input ad `x`
- `write(x)` trattata come assegnamento da `x` ad output
- Esempio:

```
{true}  
read(x);  
x := x + 1;  
write(x)  
{output = input + 1}
```

Con backward substitution:

```
x = input + 1  
x + 1 = input + 1  
input + 1 = input + 1
```

Notazione di Hoare

- Dalla verità di Claim1 e Claim2 si può dedurre Claim3

$$\frac{\text{Claim1, Claim2}}{\text{Claim3}}$$

- Utile per esprimere le relazioni fra precondizioni e postcondizioni delle strutture di controllo.
- Permette di costruire prove complesse a partire da prove più semplici

Sequenza

$$\frac{\{F1\}S1\{F2\}, \{F2\}S2\{F3\}}{\{F1\}S1;S2\{F3\}}$$

Se $F1$ e l'esecuzione di $S1$ implicano $F2$, e se $F2$ e l'esecuzione di $S2$ implicano $F3$, $F1$ e l'esecuzione successiva di $S1$ e $S2$ implicheranno $S3$.

- Esempio: da
 - $\{a+b-43 > y+7\} \ z:=a+b \ \{z-43 > y+7\}$, e
 - $\{z-43 > y+7\} \ x:=z-43 \ \{x > y+7\}$, si deduce
 - $\{a+b-43 > y+7\} \ z:=a+b; \ x:=z-43 \ \{x > y+7\}$

Istruzione condizionale

$$\frac{\{Pre \text{ and } cond\} S1 \{Post\}, \{Pre \text{ and not } cond\} S2 \{Post\}}{\{Pre\} \text{ if } cond \text{ then } S1 ; \text{ else } S2 ; \text{ end if; } \{Post\}}$$

- Se la verità di Post è garantita da:
 - Pre, la verità di cond e l'esecuzione di S1 (ramo then)
 - Pre, la falsità di cond e l'esecuzione di S2 (ramo else)

Post è garantita da Pre e dall'esecuzione dell'istruzione condizionale

Esempio

{true}

if $x \geq y$ then

$\text{max} := x$

else

$\text{max} := y$

end if

{($\text{max}=x$ or $\text{max}=y$) and
 ($\text{max}\geq x$ and $\text{max}\geq y$)}

- Ramo then:

- ($x=x$ or $x=y$) and ($x\geq x$ and $x\geq y$)
- equivalente a ($x\geq y$), implicato da Pre and cond

- Ramo else:

- ($y=x$ or $y=y$) and ($y\geq x$ and $y\geq y$)
- equivalente a $y>x$, implicato da Pre and not cond

Ciclo while

$$\frac{\{I \text{ and } \text{cond}\} S \{I\}}{\{I\} \text{ while cond loop } S; \text{ end loop}; \{I \text{ and not cond}\}}$$

- S corpo del ciclo
- I è invariante (di ciclo) se I, la verità di cond e l'esecuzione di S mantengono vero I.
- Se I è un invariante per il ciclo,
allora la verità di I all'ingresso nel ciclo while e l'esecuzione del ciclo implicano I e la falsità di cond all'(eventuale) uscita dal ciclo

Esempio

```
{x >= 0}  
while (x>0) loop  
  x:=x-1;  
end loop  
{x = 0}
```

I: $\{x \geq 0\}$ è invariante di ciclo: per backward substitution,
 $x \geq 0$ se $x-1 \geq 0$, implicata da I and cond essendo x intero.

All'uscita dal ciclo, valgono I and not cond, cioè $x \geq 0$ and $x \leq 0$, cioè
 $x = 0$

Esempio

$\{x > y\}$

while $x \neq 0$ loop

$x := x - 2$

$y := y - 2$

end loop

$\{x > y \text{ and } x = 0\}$

$x > y$ è un invariante, ma il ciclo non termina se x è inizialmente dispari o minore di 0.

Prova di correttezza di un ciclo

- Una prova di correttezza totale richiede anche la prova della terminazione del (frammento del) programma.
- La sola dimostrazione della postcondizione a partire dalla preconditione e dall'esecuzione del programma è una prova di correttezza parziale
- Non esistono regole per individuarlo; bisogna esaminare le istruzioni del ciclo
- L'invariante deve essere abbastanza forte da implicare (insieme alla condizione negata) la postcondizione del ciclo
- Deve però essere abbastanza debole da valere all'ingresso nel ciclo

Esempio: divisione intera

```
{input1 > 0 and input2 > 0}
begin
  read (x); read (y);
  div := 0;
  while x >= y loop
    div := div + 1;
    x := x - y;
  end loop;
  write (div); write (x);
end;
{input1 = output1 * input2 +
output2 and
0 <= output2 < input2 }
```

```
{input1 >= 0 and input2 > 0}
begin
  read (x); read (y); div := 0;
  while x >= y loop
    div := div + 1;
    x := x - y;
  end loop;
end;
{input1 = div * input2 + x and
0 <= x < input2 }
```

Invarianti inutili: $x \geq 0$, $div \geq 0$,
 $div > 1000$

Scelta di un invariante

- x vale inizialmente input1 e viene decrementato di y a ogni iterazione
- div vale inizialmente 0 e viene incrementato di 1 a ogni iterazione

Possibilità:

- $\text{input1} = x + \text{div} * y$
 - E' invariante e vale all'ingresso nel ciclo, ma è troppo debole
- $\text{input1} = x + \text{div} * y \text{ and } x \geq 0 \text{ and } y = \text{input2}$
 - E' invariante: con due backward substitution,
 $\text{input1} = x - y + (\text{div} + 1) * y \text{ and } x - y \geq 0 \text{ and } y = \text{input2}$, implicato da I e cond
 - E' abbastanza forte: implica la postcondizione
 - Vale all'entrata del ciclo: con backward substitution,
 $\text{input1} = \text{input1} + 0 * \text{input2} \text{ and } \text{input1} \geq 0 \text{ and } \text{input2} = \text{input2}$

Prove formali di correttezza: valutazione

Vantaggi:

- Garanzia forte della correttezza di un sistema o di una sua parte
- Hanno come sottoprodotto un'analisi informale profonda del codice, che ne migliora la comprensione

Svantaggio:

- Complesse (e costose) anche per sistemi o sottosistemi di piccole dimensioni

Appropriate se le esigenze di correttezza giustificano l'investimento.

Esecuzione simbolica

Esecuzione simbolica

- Approccio intermedio fra analisi e test
- Consiste nell'esecuzione del programma con valori simbolici anziché numerici.
- Un'esecuzione simbolica corrisponde all'esecuzione di molti test (uno per ogni possibile combinazione di valori numerici).
- I risultati possono essere usati per dimostrare proprietà del programma.

Esempio

```
read(a); read(b);  
x:=a+1;  
x:=x+b+2;  
write(x);  
[A+B+3]
```

```
a=A; b=B;  
x=A+1;  
x=A+1+B+2=A+B+3;  
output=A+B+3
```

- Notazione:
 - Convenzionalmente, il valore simbolico letto per una variabile è indicato dal suo identificatore in maiuscolo (il valore letto per x è X)
 - Fra parentesi quadre si indica il valore dell'output

Esempio: asserzione

```
{true}  
read(a);  
x:=a*a;  
x:=x+1;  
write(x);  
{output>0}
```

```
a=A;  
x=A2;  
x=A2+1;  
output=A2+1
```

Sicuramente
positiva

Esecuzione simbolica condizionale

```
read(a)
if a>0 then
  <ramo true>;
else
  <ramo false>;
end if;
```

a=A
?...

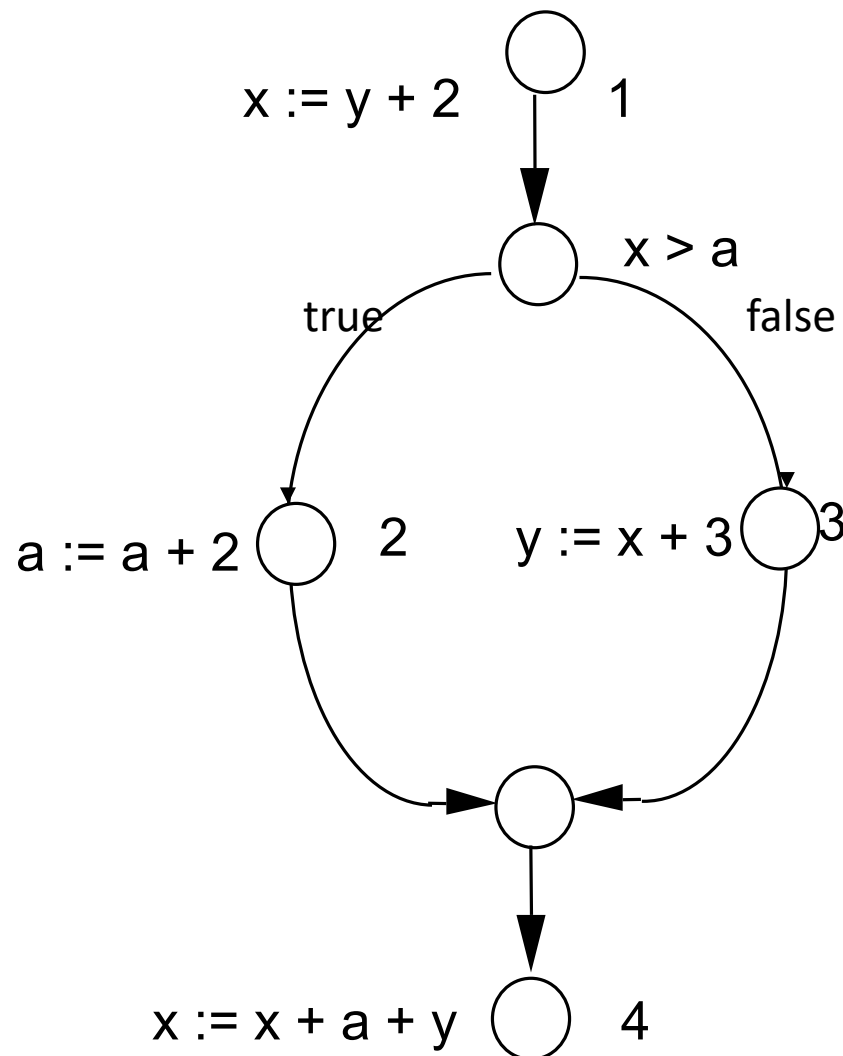
- Il valore A non contiene informazione sufficienti per scegliere il cammino

Esempio

```

x := y + 2;
if x > a then
  a := a + 2;
else
  y := x + 3;
end if;
x := x + a + y;

```



- La scelta di un cammino fra quelli possibili è determinata da una *path condition*.
- Esempio: per il ramo else, path condition è $Y + 2 \leq A$.
- Risultato dell'esecuzione:

$\langle \{a = A, y = Y + 5, x = 2 * Y + A + 7\},$	
$\langle 1, 3, 4 \rangle,$	$Y + 2 \leq A$
cammino	path condition

Esecuzione condizionata

- In generale, se P è un frammento di programma e G_P è il suo grafo di controllo, il risultato dell'esecuzione simbolica di P su un cammino è il suo *stato simbolico*, dato da:
 - valori simbolici delle variabili (equazioni del tipo identificatore=espressione simbolica, dette anche legami)
 - cammino di esecuzione (sequenza dei nodi visitati)
 - path condition (espressione logica sui valori simbolici delle variabili che porta all'esecuzione del cammino)

Regole per l'esecuzione simbolica

- Interprete: macchina che svolge l'esecuzione simbolica aggiornando lo stato simbolico
- Stato iniziale:
 - valori simbolici non definiti
 - cammino d'esecuzione nullo
 - path condition true
- read(x):
 - rimuove ogni legame per x
 - aggiunge il legame $x=X$

Regole per l'esecuzione simbolica

- `write(espressione)`:
 - `n` è un valore inizializzato a 1 e incrementato a ogni `write`
 - `output(n)` = valore simbolico di `<espressione>`
- assegnamento `x := espressione`:
 - rimuove ogni legame per `x`
 - crea un'espressione simbolica `SV` utilizzando i valori simbolici delle variabili in `<espressione>`
 - aggiunge il legame `x = SV`

Regole per l'esecuzione simbolica

- Dopo l'esecuzione dell'ultima istruzione di una sequenza corrispondente a un nodo N di G_p , il nome del nodo si aggiunge all'execution path
- if cond then s1 else s2 endif:
 - si valuta cond: eval(cond) valore simbolico
 - se eval(cond) ha un valore di verità già definito, l'esecuzione procede nel ramo corrispondente
 - altrimenti si sceglie arbitrariamente un valore:
 - true: eval(cond) si aggiunge in and alla path condition e l'esecuzione procede nel ramo true;
 - false: not eval(cond) si aggiunge in and alla path condition e l'esecuzione procede nel ramo false

Regole per l'esecuzione simbolica

- while cond loop S end loop:
 - si valuta cond: eval(cond) valore simbolico
 - se eval(cond) ha un valore di verità già definito, l'esecuzione procede nel ramo corrispondente (S o uscita dal ciclo)
 - altrimenti si sceglie arbitrariamente un valore:
 - true: eval(cond) si aggiunge in and alla path condition e si esegue S;
 - false: not eval(cond) si aggiunge in and alla path condition e l'esecuzione procede all'uscita dal ciclo
- Gli execution path e le corrispondenti path condition possono essere infiniti.

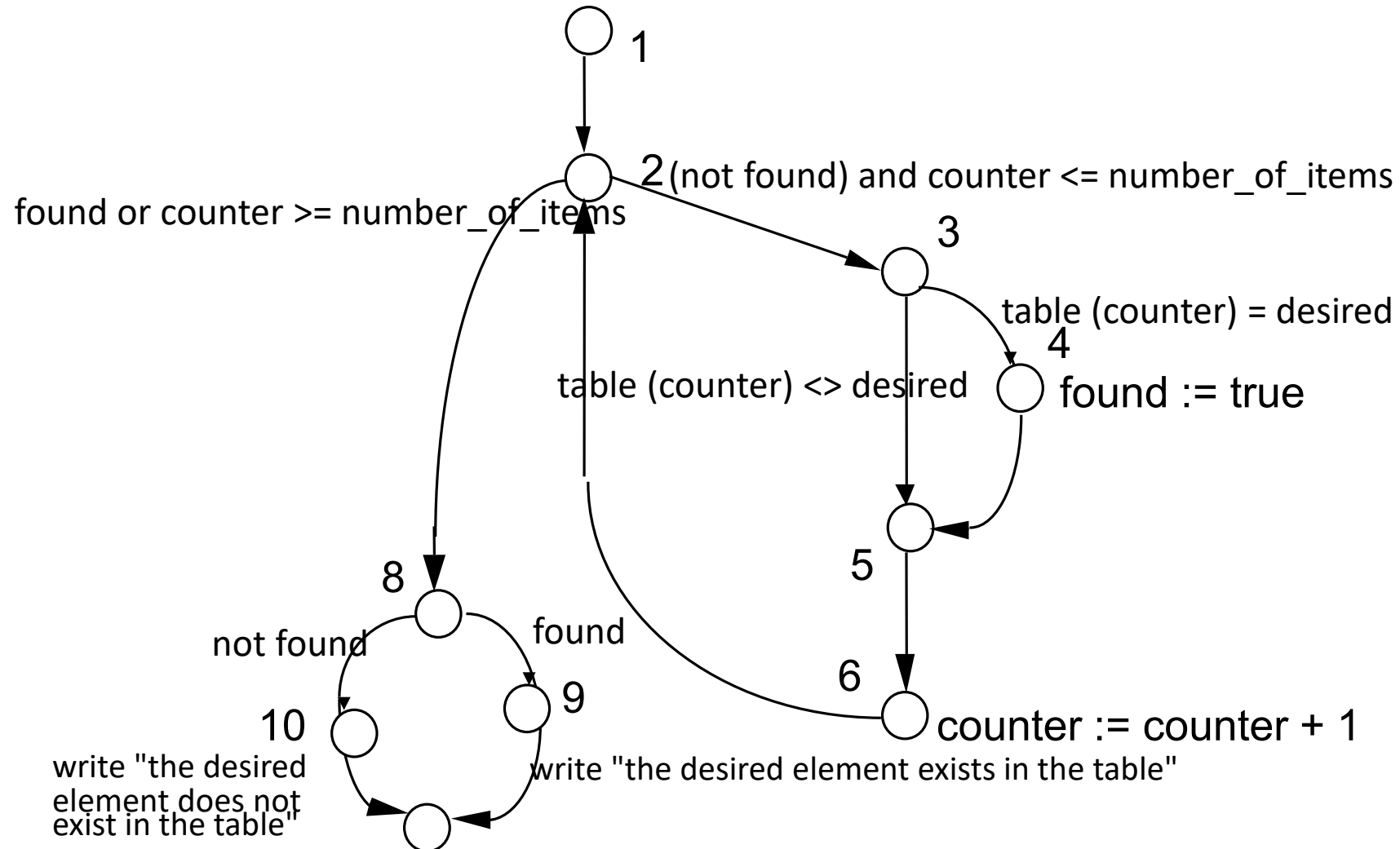
Esecuzione simbolica e test

- L'esecuzione simbolica può essere utilizzata nella selezione dei casi di test
- Una volta fissato un cammino da eseguire (per rispettare un determinato criterio) si può
 - individuare alitmicamente la path condition necessaria
 - cercare di soddisfarla con i dati di input (problema indecidibile in generale)

Esempio

1. found := false; counter := 1;
2. while (not found) and counter <= number_of_items loop
3. if table (counter) = desired_element then
4. found := true;
5. end if;
6. counter := counter + 1;
7. end loop;
8. if found then
9. write ("the desired element exists in the table");
10. else write ("the desired element does not exist in the table");
12. end if;

Grafo di controllo



Esempio

- Cammino <1,2,3,4,5,6,2,8,9>

```

1. found := false; counter := 1;
2. while (not found) and counter <=
   number_of_items loop
3.   if table (counter) = desired_element
   then
4.     found := true;
5.   end if;
6.   counter := counter + 1;
7. end loop;
8. if found then
9.   write ("the desired element exists in
   the table");
10. else write ("the desired element
   does not exist in the table");
12. end if;

```

Esecuzione	Symbolic state
Inizio	Found=?, counter=? , PC= true
1	Found = false, counter = 1, PC = true
2 (not found) and counter <= number_of_items? (true)	Found = false, counter = 1, PC = (1<=number_of_items)
3 table (counter) = desired_element ? (scelta: true)	Found = false, counter = 1, PC = (table(1) = desired_element and 1<=number_of_items)
4	Found = true, counter = 1, PC = (table(1) = desired_element and 1<=number_of_items)
5,6	Found = true, counter = 2, PC = (table(1) = desired_element and 1<=number_of_items)
2 found or counter > number_of_items? (true) 8 found? (true), 9	Found = true, counter = 2, PC = (table(1) = desired_element and 1<=number_of_items), output(1) = "the desired element exists in the table"